Applied Soft Computing 112 (2021) 107760

Contents lists available at ScienceDirect

Applied Soft Computing

journal homepage: www.elsevier.com/locate/asoc

# Constructing a stock-price forecast CNN model with gold and crude oil indicators

Yu-Chen Chen, Wen-Chen Huang *

*Department of Information Management, National Kaohsiung University of Science and Technology, 2 Juoyue Rd., Nantsu, Kaohsiung 811, Taiwan*

ARTICLE INFO

*Article history:*
Received 15 December 2020
Received in revised form 15 July 2021
Accepted 26 July 2021
Available online 30 July 2021

*Keywords:*
Stock price forecast
Deep learning
Convolutional neural networks
Long short-term memory
Bayesian optimization

ABSTRACT

In this study, we propose algorithms to predict future stock market trends based on 8 different input features, including financial technology indicators, gold prices, a gold price volatility index, crude oil price, a crude oil price volatility index, and other characteristic data using two different labeling methods with separate classification algorithms of two and three output categories, respectively including predicted stock price changes (up and down) and recommended trading actions (buy, sell, and hold), and analyze the validity of these characteristic data in terms of their ability to predict future trends. The S&P 500 (GSPC) is the target of these forecasts. Sample data from 2010 to 2018 are divided 8:2, between training and verification data, while data from 2019 are used to test the proposed approach. CNN and LSTM models are used for comparison of classification accuracy and investment returns, respectively. Bayesian optimization (BO) hyperparameters are used to improve the accuracy of the model and increase the return on investment (ROI) of the output predictions.

The purpose of this study is to verify whether using gold prices, a gold volatility index, crude oil price, and a crude oil price volatility indices as input features can enable a deep learning model accurately to predict future stock price trends, and to discuss separately the applicability of CNN and LSTM models to the abovementioned characteristics and financial indicators. We also present the results of experiments conducted to evaluate the proposed method in terms of classification accuracy and confusion matrix. In the case of three-category classification, the model takes feature data as input to outputs a predicted trading order on whether to buy, sell, or hold a given set of stocks tomorrow as well as the timing of entry and exit from each position, and also backtests the data outside the sample to find the combination of characteristics and indicators best maximizing ROI. Using this three-category method, we obtain a comprehensive ROI for a given set of individual stocks and assess whether each type of stock is suitable for the prediction model based on input features such as gold and crude oil or the fields that are suitable for the given feature.

Experimental results show that the proposed approach as able to predict whether stock price will rise or fall in the next 10 days, and the model accuracy rate can reach 67%. The results of experiments on the proposed combined CNN model with eight features, referred to as CNN8, achieved an ROI on 2019 data outside the sample period of up to 13.23%, which was superior to the 12.08% and 11.06% obtained by the models designed CNN4 (CNN with four input features) and LSTM8(LSTM with eight input features), respectively. The F1 score increased from 0.75 0.79 as a result of applying BO. The results indicate that considering the price of gold, the gold volatility index, crude oil price, and crude oil price volatility index can help obtain better ROI for companies in certain fields, such as the semiconductor, petroleum, and automotive industries, rather than merely considering financial indicators. However, for companies related to apparel, fast food, and copy processing, the input characteristics of purely financial technical indicators were found to be suitable.

© 2021 Elsevier B.V. All rights reserved.

The code (and data) in this article has been certified as Reproducible by Code Ocean: (https://codeocean.com/). More information on the Reproducibility Badge Initiative is available at https://www.elsevier.com/physical-sciences-and-engineering/computer-science/journals.

* Corresponding author.
*E-mail address:* wenh@nkust.edu.tw (W.-C. Huang).

https://doi.org/10.1016/j.asoc.2021.107760
1568-4946/© 2021 Elsevier B.V. All rights reserved.

## Code metadata

Permanent link to reproducible Capsule: https://doi.org/10.24433/CO.9139038.v1

## 1. Introduction

In recent years, an increasing number of studies have been conducted on methods using convolutional neural networks (CNN) to predict stock prices [1–3]. A significant portion of this research has focused on converting financial technology indicators into image pixels, enabling a CNN to learn to predict stock price trends. One recent study [1] found that the average return on investment (ROI) of the backtest was improved by using a CNN model to predict stock market trends, compared to that of other stock investment strategies or model prediction results considered. Furthermore, the research results of [2] demonstrated that a CNN provided better prediction results than an artificial neural network (ANN) and support vector machine (SVM) models. In contrast, this study concluded that compared to other time series models, financial indicators did not affect the CNN model positively. This could be because trends of characteristics such as general average and exponential moving average were extremely similar to closing price movement trends. Additionally, the results suggest that prediction using external factors such as gold prices and interest rates may be considered suitable.

At present financial indicators in deep learning models have shown better performance on the task of predicting stock price trends than input features as fundamental data. Therefore, numerous studies such as [1,4], and [2] have proposed that in addition to financial indicators, the feasibility and integration of additional new features should be explored. According to the results of [5], oil prices and gold prices as well as the commonly used gold and oil price volatility indices have a long-term equilibrium for the S&P 500 market and fewer short-term effects. The reason for this behavior is that investors do not often transfer assets. However, in the long run, stock market prices are affected by the economic situation and future trends. This study aims to predict the future trends of stock prices through financial technical indicators and characteristic data such as gold prices, a gold volatility index, crude oil prices, and a crude oil price volatility index. The characteristic data and stock price charts are input to a CNN model in a moving pane, and the results are compared with those of a long short-term memory (LSTM) model to identify features and methods that can accurately predict future trends to identify suitable times to buy, hold, or sell stocks. Finally, the buy, sell, and hold timings predicted by the model are used to identify the best timing to invest to achieve a good ROI.

The purpose of this study is as follows.

1. We aim to verify whether using gold price, gold volatility index, crude oil price, and crude oil price volatility index as input features of a deep learning model can accurately predict future stock price trends.
2. We further separately discuss the applicability of the abovementioned characteristics and financial indicators to the CNN and LSTM models and the classification accuracy and confusion matrix results.
3. In the case of three-category classification, we use the extracted feature data and the trained model to determine whether to buy, sell, or hold a given stock tomorrow, find the optimal timing of entry to and exit from each position, and further backtest the data outside the sample to find the combination of characteristics and indicators best maximizing ROI.
4. Using the three-category method, we obtain the ROI of individual stocks over a time period outside the sample and assess whether each type of stock is suitable to be detected with input features such as gold and crude oil prices or fields that are suitable for the given feature.

In this study, we use two different target labeling methods to predict and classify future stock market trends, including a method with two categories (up and down) and a method with three categories (buy, sell, and hold) based on financial technology indicators and gold prices, a gold volatility index, crude oil prices, a crude oil price volatility index, and other characteristic data , and analyze the validity of these characteristic data corresponding to future trends. The CNN and LSTM models were used to compare the classification accuracies and investment returns, respectively.

The contributions of this research are as follows.

1. Using the recorded values for the price of gold, a gold volatility index, crude oil prices, and a crude oil price volatility index as input features of a deep learning model, our proposed model demonstrated the ability to predict whether stock price will rise or fall over the next 10 days, achieving an accuracy rate of 67%.
2. We found that of the two types of features explored, in the two-category classification model, the classification accuracy of the CNN and LSTM models was indistinguishable, whereas in the three-category classification model, the classification results of using both together proved more accurate.
3. Measurement of the combination model (CNN8) shows that the ROI for 2019 data, outside the sample set, reached up to 13.23%, which was superior to the 12.08% and 11.06% obtained by the CNN4 and LSTM8 models, respectively.
4. Considering gold price, a gold volatility index, crude oil prices, and a crude oil price volatility index is shown to help obtain better ROI for companies in the semiconductors, petroleum, and automotive industries, as opposed to merely considering financial indicators. However, for companies related to apparel, fast food, and copy processing, the input characteristics were suitable for prediction on the basis of pure financial technical indicators.

## 2. Background

Stock trend or price prediction has long been a popular research topic. Recently, numerous new techniques such as machine learning [6,7], deep learning frameworks [8–12], fuzzy/rough sets [13,14], sentiment analysis [15,16], genetic algorithms [17], Markov decision processes [18], and entropy [19] have been implemented. [20] explores the impact factors of virtual online stock message boards in terms of past performance. Analysis of stock market data has also been performed using candlestick regression [21], dimension reduction [22], fundamental and technical analysis [23], a warning system [24], and stock returns [25–27]. Deep learning models have been widely applied in a diverse array of fields; indeed, various implementations have been developed for such applications as path tracking and control of UAVs (Unmanned Aerial Vehicle) [28,29] and automated detection and diagnosis of COVID-19 from x-ray images [30] .

A recent study attempted to predict the direction of price changes in Brazilś Sao Paulo Index (IBovespa) every 15 min [31]. They used up to 175 types of technical indicators from technical analysis libraries (TA-Lib). They used a variety of deep learning and machine learning algorithms such as simple investment strategies, multi-layer perceptrons (MLPs), random forest classifiers, and LSTM models to predict future trends and compare

their results. Their results demonstrated that the LSTM models provided superior performance compared to the other models in terms of prediction accuracy and obtained the best ROI on each backtesting transaction. Conclusively, technical analysis combined with an LSTM model may be considered to have a certain reference value for investors to predict and classify future stock trends. Moreover, LSTM networks with decomposition methods and a Grey Wolf Optimizer were used to capture nonlinear features of wind speed time series [32].

The purpose of Bayesian optimization (BO) [33] is to find and adjust the hyperparameters of a given model for effective performance on the validation dataset. This optimization method is understood to be superior to random or grid searches. To use BO, we need an efficient method to model the distribution of the objective function. The proposed approach uses Gaussian processes (GPs) to establish a probability model of an objective function. In another sampling process, more promising hyperparameters are identified to evaluate the real objective function, and prior knowledge is used to approximate a posterior distribution of the unknown objective function, to adjust the hyperparameters. This method can identify better hyperparameter settings than a random search within fewer iterations.

Fifteen technical indicators were used to generate 15 x 15 images as learning data for a CNN [1]. Their results showed that the annualized return of their proposed method CMM-TA after the backtest was significantly superior to that of the other methods compared, such as long-term hold (buy and hold, or BaH), simple moving average (SMA), relative strength index (RSI), LSTM models, and MLP. Multi-input LSTM (MI-LSTM) [34] used additional LSTM logic to construct a mainstream, which enabled the model to determine more useful reservations from less relevant information by filtering noise. This architecture demonstrated superior prediction performance compared to the original LSTM prediction. The bullish and bearish (True & False) [35] results produced using technical indicators were more useful than the technical indicators themselves, and random forest prediction was the most commonly used machine learning technique. Another study used 175 technical indicators to construct datasets [31]. Their LSTM prediction method reduced error compared to other machine learning methods considered, and had the highest ROI.

A new forecasting model [36] was recently developed based on support vector regression (SVR) with a wrapper-based feature selection approach using multi-objective optimization technique to deal with this challenge. In that model, features based on technical indicators such as simple moving average (SMA) and exponential moving average (EMA), and Kaufmanś adaptive moving average (KAMA) were utilized. SMA, EMA, and KAMA indicators were obtained from the Brent crude oil closing prices under different parameters. The use of closing price and nine technical indicators as input features for a CNN model was proposed [2]. The S&P 500 stock basket was adopted as a dataset, and the classification accuracy rate results were compared with those of ANN and SVM models. They not only confirmed that using manual methods to change particular hyperparameters can make CNN prediction classification more accurate but also shows that the input characteristics of financial indicators can be used to classify future stock prices using the proposed architecture. Furthermore, it was confirmed that their proposed method was more accurate than ANN and SVM models on the task of stock trend prediction.

To study the effect of using different combinations of financial indicators and oil prices, gold prices, a gold price volatility index, and an oil price volatility index with S&P 500 stocks, experiments were conducted using the abovementioned CNN and LSTM models. To this end, this study uses different target data labeling methods, including one of two categories (up, down) and one of three categories (buy, sell, hold) to discuss the experimental results. Furthermore, Bayesian methods were used to optimize hyperparameters and update classification weights to improve on the accuracy of prior models and increase their ROI. In summary, Table 1 lists the various proposed prediction models and their training datasets.

**Table 1**
Prediction models and training dataset.

| Reference | Year | Prediction models | Training dataset |
|---|---|---|---|
| [1] | 2018 | CNN, BaH, RSI, SMA, LSTM | 15 technical indices |
| [34] | 2018 | MI-LSTM, LSTM, RNN | 300 historical stock price |
| [35] | 2015 | ANN, SVM, RF, NB | 10 technical index |
| [31] | 2017 | ANN, RF, LSTM | 175 technical index |
| [2] | 2019 | CNN, ANN, SVM, | 10 technical index |
| [36] | 2020 | SMA, EMA, KAMA, SVM, PSO | 21 technical index |

## 3. Research methods

This section explains the prediction target and marking methods of the proposed two- (up and down) and three-category (buy, sell, hold) models. It further explains the relationship between oil, gold, and stocks, as well as a calculation method of the data source and characteristic data. Additionally, the architecture and parameters of the proposed deep learning model, such as the selection of model design variables and hyperparameters, are included. Finally, we explore methods to optimize classification weights and ROIs when the training samples are unbalanced.

### 3.1. Prediction target

The reason for choosing the S&P 500 (GSPC) index is that it remains the second largest stock price index in the United States. The impact of gold and oil prices on the index is even more significant, and the investment risks associated with it are better-diversified, reflecting the variability of a wide range of included markets. The results of [5] showed that oil prices, gold prices, gold price volatility indices, and oil price volatility indices had long-term equilibrium effects on the S&P 500 market. Therefore, the objective of this research is to predict future trend of the broad market index through financial technical indicators, gold prices, a gold volatility index, crude oil prices, a crude oil price fluctuation index, and other characteristic data.

### 3.2. Classification

This subsection explains the marking methods of the classification of the research data. There are two target labeling methods: for two categories (up, down) and three categories (buy, sell, hold). Their detailed descriptions are as follows.

#### 3.2.1. Two-category marking method

Characteristic data such as oil and gold affect the index trend in the long run; therefore, a 10-day period was used as the prediction target date in this study. Two types of trend were considered, including rising and falling types. Furthermore, we consider if an extracted trend is actually used in a futures index, stock market, or index-type securities investment trust fund, a commission will be charged for the transactions. Therefore, considering the transaction fee, the future trend [2] is defined as follows. If the closing price after 10 days divided by today's closing price exceeded 1.007, the future trend was judged to be rising. Otherwise, the future trend of the index was judged to be falling. The algorithm used in the proposed two-category classification model is outlined in Algorithm 1.

**Table 2**
Data set.

| Code | Data set | Interval | Start date | End date | Total items |
|---|---|---|---|---|---|
| GSPC | S&P 500 | Daily | 2010/01/01 | 2019/12/31 | 2493 |
| CL = F | Crude oil price | Daily | 2010/01/01 | 2019/12/31 | 2493 |
| OVX | Crude oil volatility index | Daily | 2010/01/01 | 2019/12/31 | 2493 |
| GC = F | Gold price | Daily | 2010/01/01 | 2019/12/31 | 2493 |
| GVZ | Gold volatility index | Daily | 2010/01/01 | 2019/12/31 | 2493 |

**Algorithm 1:** Future trend computation of two-category classification model

```
Data: cp_today, cp_10
Result: Find the future trend ft.
cp_today = Today's closing price;
cp_10 = Closing price after 10 days;
float fraction = cp_10 / cp_today;
if fraction > 1.007 then
    ft = 0 (up);
else
    ft = 1 (down);
end
```

*3.2.2. Three-category marking method*

The three-category marking method refers to the labeling algorithm [1]. The implementation of Algorithm 2 was implemented using Python code. Each observation of the closing price used an 11-day window size, and each moving step was 1. If the closing price of the middlemost day of the 11 days (the 6th day) was the highest, the last day (day 11) was marked as 2 (sell). If the closing price of the middlemost day was the lowest, the 11th day was marked as 0 (buy). Otherwise, the last day was marked as 1 (hold). By marking the data in this manner, a stock can be bought at the lowest and sold at the highest possible price.

*3.3. Data set*

Table 2 shows the details of the dataset used in the experiments conducted. The S&P 500 (GSPC) dataset was sourced from Yahoo Finance; it featured daily opening, lowest, highest, closing, and adjusted closing prices. Yahoo Finance was also the data source for information used on crude oil price (CL = F) and gold price (GC = F); however, these refer only to the daily closing prices. The crude oil volatility and gold volatility indices were obtained from the CBOE official website, including the gold available since 2010 (GVZ) and petroleum (OVX) series volatility index. The overall dataset of this study covers the daily price and index values from January 2010 to the end of December 2019. The dataset includes the corresponding daily time series data, with a total of 2493 observations for each code.

*3.4. Model input features*

The structures of CNN1–CNN4 in Table 3 were selected and constructed in accordance with [2]. The proposed CNN5–CNN8 architectures list in Table 3 were constructed as well. The source of the dataset used is provided in Section 3.3.

Algorithm 3 shows a feature image and the corresponding class-labeling method. The second line of the algorithm sets the time period to 20 days. The third line shows the possible classification action after one day, including buy, sell, or hold. The seventh line imports all features contained in the dataset. The eighth to twelfth lines contain a for loop; for each iteration, the loop adds the features from the *i*th to the i+time_period day

**Algorithm 2:** Three-category marking method

```
1  def create_label(df, col_name, window_size=11):
2      row_counter = 0
3      total_rows = len(df)
4      labels= np.zeros(total_rows)
5      labels[:] = np.nan
6      while row_counter < total_rows do
7          if row_counter >= window_size − 1 then
8              window_begin = row_counter-(window_size - 1)
9              window_end = row_counter
10             window_middle = (window_begin + window_end) /
               2
11             min_ = np.inf
12             min_index = -1
13             for i in range(window_begin, window_end + 1) do
14                 price = df.iloc[i][col_name]
15                 if price < min_ then
16                     min_ = price
17                     min_index = i
18                 else if price > max_ then
19                     max_ = price
20                     max_index = i
21             end
22             if max_index == window_middle then
23                 labels[row_counter] = 2
24             else if min_index == window_middle then
25                 labels[row_counter] = 0
26             else
27                 labels[row_counter] = 1
28         row_counter = row_counter + 1
29     end
30     return labels
```

and includes it in the list of cnn_x. Then, the classification item of cnn_y corresponding to cnn_x is the classification mark for the next day. For the marker generation method, please refer to Section 3.2.2. Fig. 1 presents feature images drawn by CNN8 after observing the 14 input variables within a 20-day period. The feature maps are displayed using a random number method to facilitate visualization of different classification images. The top of each feature map shows the image number of the feature map and its corresponding class tags, where Buy = 0, Hold = 1, Sell = 2.

*3.5. Model parameter design and optimization*

Deep learning models generally involve two types of parameters, including model design variables and hyperparameters. The model design variables include a type of optimizer, loss function, and activation function. A hyperparameter is a parameter whose value is used to control the learning process. This includes learning rate, dropout, and batch size.

Parameter selection of a deep learning model is an essential process. In this study, the best values of the learning rate of the optimizer (learning rate), dropout of neurons in the neural

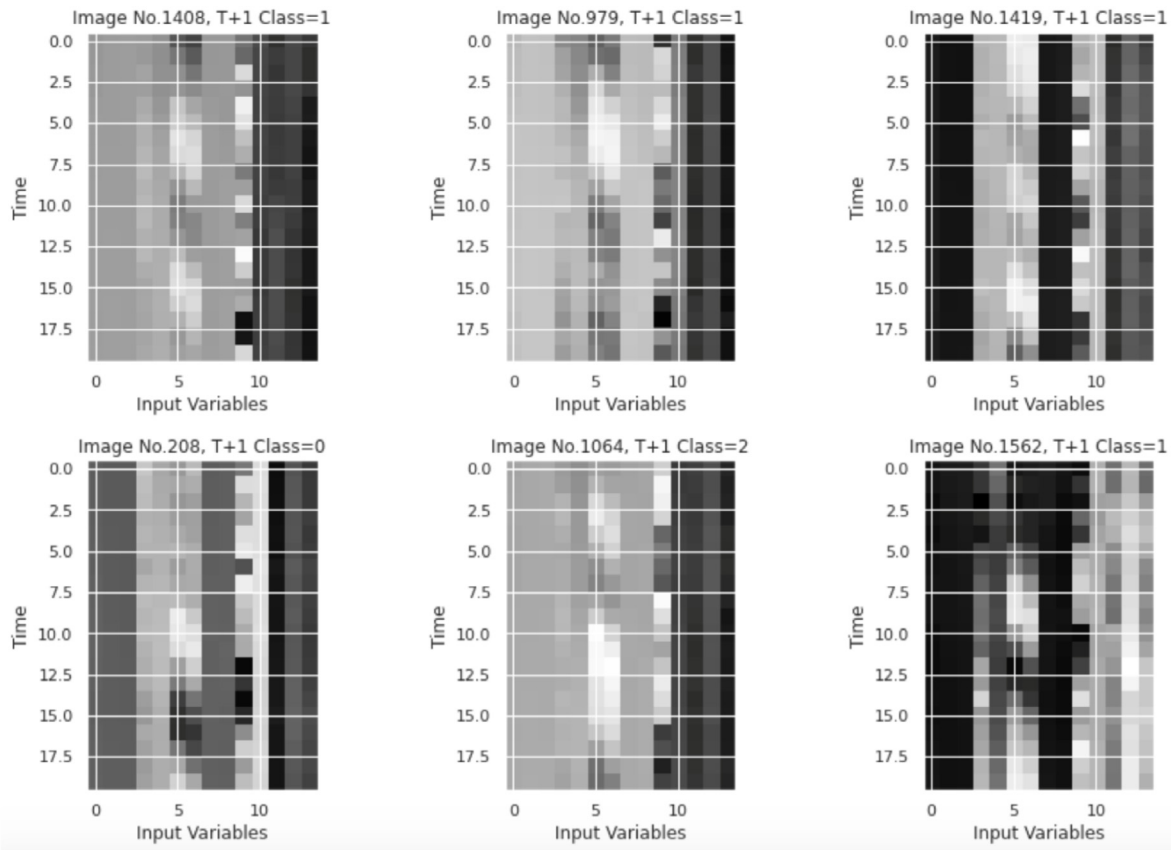


**Fig. 1.** CNN8 three-dimensional feature data map.

**Table 3**
Model input features.

| Model name | Input variables |
|---|---|
| CNN1 | Closing price |
| CNN2 | Closing price, SMA, EMA |
| CNN3 | Closing price, SMA, EMA, ROC, MACD |
| CNN4 LSTM4 | Closing Price, SMA, EMA, ROC, MACD, Fast %K, Slow %D, Upper band, Lower band, %B |
| CNN5 | Closing price, oil price, oil volatility index |
| CNN6 | Closing price, gold price, gold volatility index |
| CNN7 LSTM7 | Closing price, gold price, gold volatility index, oil price, oil volatility index |
| CNN8 | Closing price, SMA, EMA, ROC, MACD, Fast %K, Slow %D, Upper band, Lower band, %B, oil price, oil volatility index, gold price, gold volatility index |

**Table 4**
Hyperparameter value search range.

| Parameter | Minimum value | Maximum values |
|---|---|---|
| Dropout rate1 | 0 | 0.5 |
| Dropout rate2 | 0 | 0.5 |
| Learning rate | 1e−5 | 1e−2 |
| Batch size | 16 | 128 |

network (dropout), and batch size (batch size) were determined. Using BO to determine the best batch size value was beneficial in increasing the speed of the training without alleviating the effect of several trainings. Using BO for dropout was also found to be helpful in randomly discard some neurons to prevent the model from overfitting. Furthermore, using BO to find the best discard ratio also prevents overfitting or discarding an excessively large number of neurons. Similarly, the use of BO for the learning rate controls the model learning progress. A very small value of the learning rate causes the training process to be considerably slow, whereas a very high value may cause failure to converge. Adjustment using BO is performed to find the optimal value of the learning rate so that it can converge normally. Table 4 lists the initial ranges used for finding the hyperparameter values.

**Algorithm 3:** Generating feature map and corresponding class label

```
from keras.utils import np_utils
time_period = 20
day = 1
cnn_x = []
cnn_y = []
indexs = []
dataset_scaled_x = dataset_scaled[feature_names]
for i in tqdm.tqdm_notebook(range(0, len(dataset_scaled) - time_period - 1 -day)) do
    cnn_x.append(dataset_scaled_x.iloc[i:i+time_period].values)
    r = dataset_scaled['return'].iloc[i+time_period -1 + days]
    rr = np_utils.to_categorical(r, num_classes = 3)
    indexs.append(dataset_scaled.index[i + time_period - 1]
end
```

### 3.6. Model architecture and parameters

This section illustrates the proposed architecture, and provided detailed descriptions of the CNN mode and LSTM architectures.

**Table 5**
The CNN model parameters.

| Layer (type) | Output shape | Number of parameters |
|---|---|---|
| conv2d_2 (Conv2D) | (None, 20, 14, 32) | 896 |
| conv2d_3 (Conv2D) | (None, 20, 14, 64) | 18496 |
| max_pooling2d_1 (MaxPooling2) | (None, 10, 7, 64) | 0 |
| dropout_1 (Dropout) | (None, 10, 7, 64) | 0 |
| flatten_1 (Flatten) | (None, 4480) | 0 |
| dense_2 (Dense) | (None, 128) | 573568 |
| dropout_2 (Dropout) | (None, 128) | 0 |
| dense_3 (Dense) | (None, 3) | 387 |
| Total params: 593,347 | Trainable params: 593,347 | Non-trainable params: 0 |

**Table 6**
The LSTM model parameters.

| Layer (type) | Output shape | Number of parameters |
|---|---|---|
| lstm (LSTM) | (None, 20, 128) | 73216 |
| dropout_1 (Dropout) | (None, 20,128) | 0 |
| lstm_1 (LSTM) | (None, 128) | 131584 |
| dropout_2 (Dropout) | (None, 128) | 0 |
| dense (Dense) | (None, 32) | 4128 |
| dense_1 (Dense) | (None, 3) | 99 |
| Total params: 209,027 | Trainable params: 209,027 | Non-trainable params: 0 |

#### 3.6.1. CNN model architecture

Figs. 2 and Table 5 depict the CNN model architecture and parameters used in this study; the model architecture was extended in accordance with [1]. First, two convolutional layers were used to convolve the original image with a specific filter to generate new feature indices. The number of filters assigned was 32 and 64, and the kernel_size was 3∗3. To obtain different filter effects, each filter generated different convolution operations in a random processing mode. Subsequently, max-pooling was used to select the largest value in the convolutional matrix. A pool_size value of 2∗2 was used to set the reduction ratio to compress the images and retain important information. To prevent overfitting, dropout was used to remove some neurons, and finally, a fully connected layer was used to flatten and connect the neurons to the next layer, and voting was performed to classify the extracted feature information. Among these mechanisms, dropout 1 and dropout 2 were not always be fixed at a certain value. Thus, BO was used to find the best value. The output layer was the second and third in the two-category and three-category classification models, respectively. A softmax activation function was used in the output layer. The solid optimizer selected the Adam optimizer. Moreover, as the output result is a classification; therefore, the loss function determined the cross-entropy error (CEE).

#### 3.6.2. LSTM model architecture

Table 6 presents the LSTM model parameters used in this study. First, a 2-unit 1-layer LSTM with a dropout layer was used. An LSTM architecture is adopted to solve the problems of gradient disappearance and gradient explosion during long sequence training. It controls the transmission state through the gated state, remembers the need for continuous memory, and forgets unimportant information. The dropout layer drops some neurons to prevent overfitting, and eventually connect to the next layer of the deep neural network. Finally, the model uses a voting mechanism to check the category of a feature. Dropout 1 and dropout 2 were not always fixed to a certain value. Hence, BO was used to find the best value. The output layer was the second and third in the two-category and three-category classifications, respectively. A softmax activation function was used for the output layer. The solid optimizer selected an Adam optimizer. Moreover, classification predictions were given as output results; therefore, a CEE loss function was used.

### 3.7. Three-category weight optimization method

For a given marked point with a window size of 11, there were 147 real-marked buying points, 163 selling points, and 2150 holding points between the years 2010 and 2019. Such an unbalanced sample caused some difficulty in training the model, because if a model weight ratio was not included, the default value was regarded as 1:1:1. In this case, the model observes each image fairly; moreover, there were many hold categories; therefore, a large number of observation points was held. Finally, the image may be expected to mistakenly attribute the result to the holding point. Algorithm 4 presents the code used for calculating classification weights. Line 1 of the code imports the built-in function library compute_class_weight from Scikit-learn (https://scikit-learn.org/stable/). The aim of this function is to calculate weights to balance the classes in the input samples. Line 5 determines whether the class_weight in y is considered "balanced"; if yes, no action is returned, otherwise, the weight is calculated. The source code of this function can be obtained from the Scikit-learn support vector classifier (SVC). Lines 8, 9, and 10 calculate the sample_weight, which is the array generated by class_weight corresponding to the result of y.

In summary, this study updates the classification weight ratio; however, while designing the weights, it is not possible to refer to data outside the training set. The second line in Algorithm 4 indicates that only the cnn_y_train is simply input, i.e., only the training set data are dropped. The value_counts in Algorithm 5 shows that the training set contained a total of 107 buying points, 119 selling points, and 1528 holding points. The Python built-in library function compute_class_weight (Algorithm 4) is used to calculate the classification weight ratio; the results are shown in Algorithm 5. The weights of the buying, selling, and holding points were 5.46, 4.91, and 0.38, respectively. These are represented as follows.

```
  Buying point (Buy) = mark category as 0 => weight = 5.46417445
         Hold (Sell) = mark category as 1 => weight = 0.38263525
Selling point (Hold) = mark category as 2 => weight = 4.91316527
```

### 3.8. ROI algorithm

In this section, the formulas used to calculate the ROI are presented, and the transaction code is explained in detail. This section is divided into two subsections, including a calculation formula for the rate of return and the description of the trading code.

#### 3.8.1. Formula to calculate the rate of return

Eqs. (1)–(3) were used to calculate the rate of return on investment in this study. The investment rate of return is the ratio of investment profit to invested capital. This value can be negative. The annualized rate of return used in the study is relatively high compared to the rates of return for different time periods.

$$ROI(\%) = \frac{Net\ investment\ gains\ and\ losses}{Total\ investment\ cost} \tag{1}$$

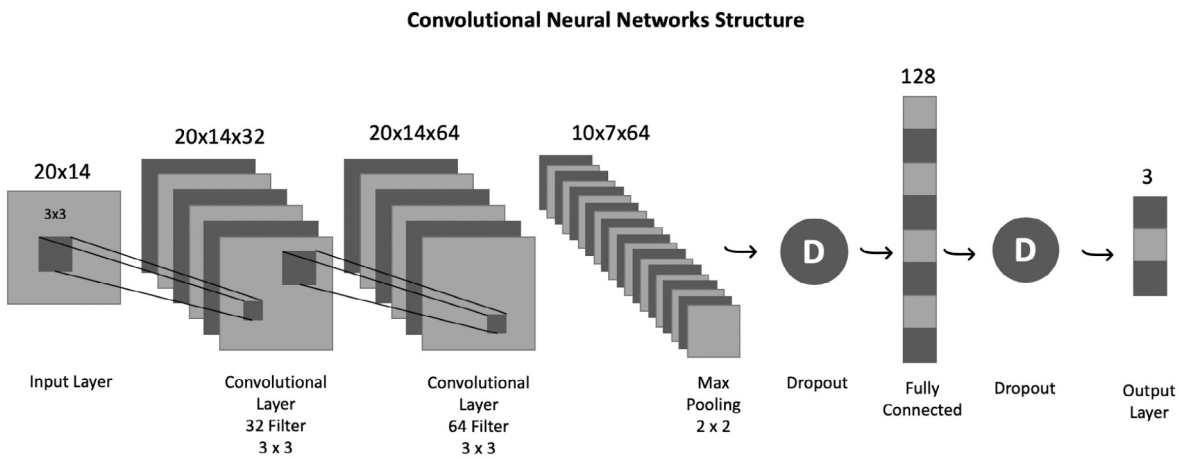


**Fig. 2.** CNN model architecture.

**Algorithm 4:** Calculate classification weights on three-category classification

```
from sklearn.utils.class_weight import compute_class_weight
import tensorflow as tf
def get_sample_weights(y):
    y = y.astype(int)
    class_weight = compute_class_weight('balanced', np.unique(y), y)
    sample_weights = y.copy().astype(float)
    for i in np.unique(y) do
        sample_weights[sample_weight == i] = class_weight[i]
end
    return class_weight, sample_weights
```

**Algorithm 5:** Show the results of classification weight calculation

```
label = np.argmax(cnn_y_train, axis=1)
label = np.array(label)
class_weight, sample_weight = get_sample_weights(label)
real class weights is [5.46 0.38 4.91] [0 1 2]
value_counts(array([0,1,2]), array([107, 1528, 119]))
```

$$Total\ return(\%) = \frac{Total\ net\ investment\ gains\ and\ losses}{Total\ investment\ cost}, \quad (2)$$

$$Annualized\ rate\ of\ return(\%) = (Total\ return + 1)^{\frac{1}{number\ of\ years}} - 1, . \quad (3)$$

*3.8.2. Transaction code description*

Algorithm 6 illustrates a dataset of trading signals and prices. The price in the data frame is the real price, while the signal is a trading signal generated by the prediction, both of which are placed at the corresponding time points. Algorithm 7 presents a formalization of the algorithm used for calculating ROI in this study. The initial capital was set to USD 10,000, and the transaction fee for each transaction was 0.09%. The transaction fee refers to the eToro platform. The variable stock in line 5 records current holdings in terms of number of shares. As per the comparison in lines 9 and 13, if the value of the signal was 1, no action was taken while the stock was held. If the value was 0, a buy action is performed; if it was 2, a sell action is performed. The transaction fee was then deducted for each transaction. After the transaction is complete, the number of shares held was multiplied by their closing price on the last day. This was then used to calculate ROI in addition to the balance on hand.

**Algorithm 6:** Trading signal and price dataset

```
1 S_dataset = pd.DataFrame({
2         'price': a,
3         'signal': b,
4 })
```

**Algorithm 7:** Calculation method for ROI

```
1  procedure ROI()
2      money = US$10000
3      handling_fee = 0.0009
4      total_money = money
5      for i in range(len(S_dataset)) do
6          signal = S_dataset['signal'][i]
7          price = S_dataset['price'][i]
8          if signal == 0 then
9              if total_money > price then
10                 stock = int(total_money/price)
11                 money = total_money − (stock ∗ price) + (stock ∗
                   price ∗ handling_fee)
12         else if signal == 2 then
13             if numberofholding ≠ 0 then
14                 money = total_money + (stock ∗ price) − (stock ∗
                   price ∗ handling_fee)
15                 stock = 0
16 end
17
   lastdaymoney = S_dataset['price'][len(S_dataset) − 1] ∗ stock
18     total =
   (total_money + lastdaymoney) − (lastdaymoney ∗ handling_fee)
19     ROI = ((total-10000)/10000) * 100
```

## 4. Experimental results and analysis

The proposed two- and three-category marking methods serve as a basis to discuss the validity of financial technology indicators and characteristic data such as gold price, gold volatility indices, crude oil price, crude oil price, and volatility indices. Furthermore, we used the model accuracy rate to evaluate the performance of each indicator. Subsequently, we verified the applicability of

**Table 7**
Feature accuracy comparison.

| Feature | Accuracy | Precision | Recall | F1 score |
|---|---|---|---|---|
| CNN1 | 0.57 | 0.57 | **1.00** | 0.72 |
| CNN2 | **0.68** | 0.68 | **1.00** | **0.81** |
| CNN3 | 0.52 | 0.71 | 0.59 | 0.64 |
| CNN4 | 0.60 | **0.77** | 0.64 | 0.70 |
| CNN5 | 0.51 | 0.70 | 0.58 | 0.63 |
| CNN6 | 0.58 | 0.72 | 0.70 | 0.71 |
| CNN7 | 0.62 | 0.71 | 0.80 | 0.75 |

these feature data and classification methods to CNNs and LSTM models. The hyperparameters were optimized using Bayesian neural networks, and the investment rate of return is calculated.

### 4.1. Two-category classification experiment results

In this classification, the data were marked in a two-category manner (up and down), and the S&P 500 (GSPC) was the forecast target. The test data were recorded in 2019, which lies outside the time period of the sample. The sample data from 2010 to 2018 was split 8:2, where 80% of the sample data were used as training data and the remaining 20% were used as verification data. The following three subsections provide further details.

#### 4.1.1. Test dataset to evaluate feature validity

Table 7 presents the experimental results in the case in which the hyperparameters were not optimized based on the same model architecture. In this table, the recall values of CNN1 and CNN2 were 1. Therefore, it was determined that the models did not correctly learn to classify the input dataset. In case of CNN1, this was attributed to a considerably lower number of features. Moreover, in CNN2, the two financial indicators of SMA and EMA moved significantly nearer the closing price; therefore, several similar feature images were generated. However, some of the same feature images were marked as 0, whereas others were marked as 1; as a result, the model became unable to learn correctly. Although CNN2, CNN5, and CNN6 included three input features, owing to the negative relationship between the characteristics of gold and oil and the closing price, strong features were generated. The results of CNN4 and CNN7 show that applications using CNNs to predict future long-term stock market trends, diverse financial indicators and characteristic data such as gold prices, gold volatility indices, crude oil prices, and crude oil price volatility indices are effective. The model training accuracy is shown in Fig. 3.

#### 4.1.2. Test set evaluation model

The results in Table 8 indicate that for the two-category classification method, the technical indicators were more suitable for LSTM networks because these characteristics were usually proportional to the stock price. The other characteristic data were more suitable than financial indicators for use in CNNs. Furthermore, the price of gold and the gold volatility index were inversely proportional to the stock prices; therefore, it is inferred that the relationship between these features and the stock price forms a strong characteristic, whereas the crude oil price and crude oil price volatility index are usually more strongly affected by stock prices.

Three familiar machine learning models (MARS, GMDH, DENFIS), and LSTM-ALO are also implemented for comparison with the proposed method. For easier references, the links to the implemented source codes are attached. MAR (Multivariate Adaptive Regression Splines) method [37,38] was implemented, the accuracy is 0.4, and the google collab implemented source code is shown in https://colab.research.google.com/drive/1ivma1ei6looj82Rq3MqUBZ_Wp18k6mKg?usp=sharing. Multivariate Adaptive Regression Splines (MARS) [39] is a method for flexible modeling of high dimensional data. The model takes the form of an expansion in product spline basis functions, where the number of basis functions, as well as the parameters associated with each one (product degree and knot locations), are automatically determined by the data [40].

GMDH (Group Method of Data Handling) [41–43] method is implemented to find the accuracy to be 0.3967. The google collab implemented source code is shown in [GMDH method] https://colab.research.google.com/drive/1-B6BXi79gmIHvHpvnP34zVvdedGf0vXU?usp=sharing. GMDH algorithm uses polynomial support functions. The general connection between input and output variables can be found in the form of a functional Volterra series [44]. The structure of the Multilayer GMDH algorithm is very similar to that of multilayer feedforward neural networks but the number of layers, as well as the number of nodes, is objectively defined by an external criterion [45].

DENFIS (dynamic evolving neural-fuzzy inference system) method [46] is used R package "frbs" [47] to find the accuracy is 0.4258, and the R implemented source code is attached to the link below [DENFIS method] https://drive.google.com/file/d/15gjnrvestk09DylxL_1ex8d2z6dyhgT0/view. frbs [48] is an R package which is aimed to construct models from data based on fuzzy rule-based systems (FRBSs) by employing learning procedures from Computational Intelligence (e.g., neural networks and genetic algorithms) to tackle classification and regression problems. DENFIS [46] evolves through incremental, hybrid (supervised/unsupervised), learning, and accommodates new input data, including new features, new classes, etc., through local element tuning. New fuzzy rules are created and updated during the operation of the system. At each time moment, the output of DENFIS is calculated through a fuzzy inference system based on most activated fuzzy rules which are dynamically chosen from a fuzzy rule set.

The ALO (Ant Lion Optimizer) algorithm is first proposed by [49] and is a gradient-free algorithm. It can avoid local optima by using the random walk and the roulette wheel. The ALO also has very few parameters to adjust and adaptively decrease the ant's movement, making the convergence quickly and to be more robust. So far, it has been applied to several applications, such as for solving multi-objective optimization problems [50], for optimal location and sizing of renewable distributed generations [51], for feature selections [52], and various applications [53]. By implementing the LSTM-ALO [54–56] algorithm, the learning rate with a lower bound of 1e-7 and the upper bound of 1, is optimized to find the minimum loss. The learning rate is found to be 0.038, the loss equals 0.65, and the overall accuracy is about 0.62. The implemented google collab code is shown in https://colab.research.google.com/drive/1_haElpiFcNLIqDqBhWzAlvp1TDPRa4AG?usp=sharing. From Table 8, it is clear to see that LSTM-ALO is higher than LSTM7 but less than LSTM4, and almost equals to CNN7 by the value of accuracy.

#### 4.1.3. Bayesian hyperparameter optimization

The hyperparameters for CNN7 in Table 9 appear before the optimization, where dropout rate1 was 0.25, dropout rate2 was 0.5, learning rate was 0.001, and batch size was 100. B-CNN7 is a result of optimization in this study. The dropout rate1 was 0.164, dropout rate2 was 0.199, the learning rate is adjusted to 0.001, and the batch size was 89.

Using BO to optimize the hyperparameters, the learning rate, dropout, and batch size achieved better prediction results than the original CNN7. This indicates that the use of hyperparameter

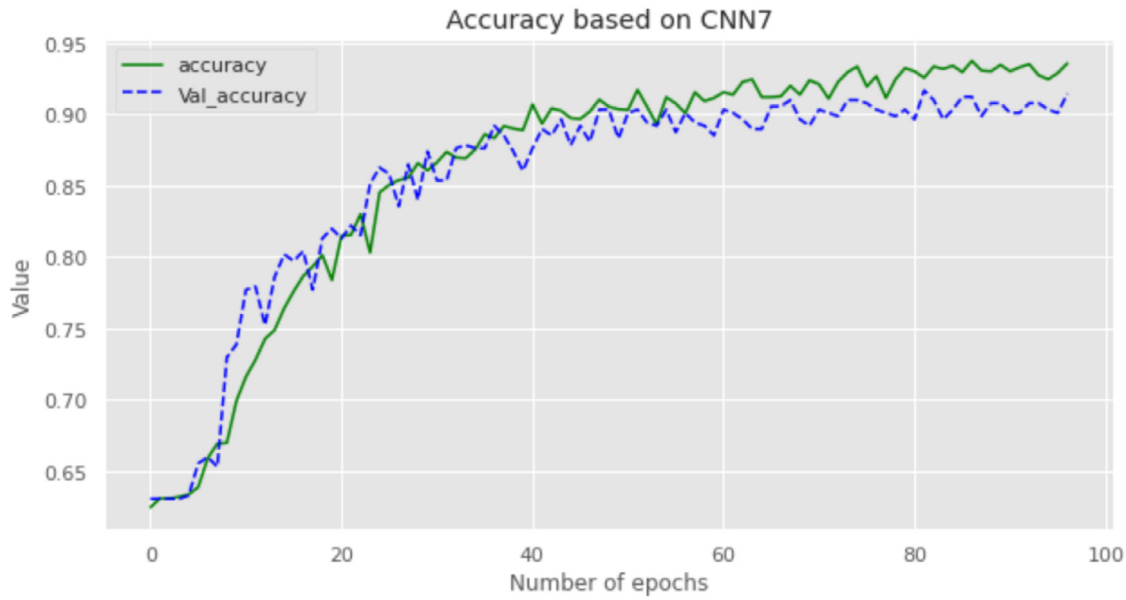


**Fig. 3.** CNN7 model training accuracy.

**Table 8**
Comparison of LSTM and CNN accuracy with other methods in two-category classification.

| Feature | Accuracy | Precision | Recall | F1 score |
|---|---|---|---|---|
| CNN4 | 0.60 | **0.77** | 0.64 | 0.70 |
| LSTM4 | **0.68** | 0.72 | **0.93** | **0.81** |
| CNN7 | 0.62 | 0.71 | 0.80 | 0.75 |
| LSTM7 | 0.54 | 0.69 | 0.68 | 0.68 |
| MARS [40] | 0.4 | – | – | – |
| GMDH [43] | 0.3967 | – | – | – |
| DENFIS [46] | 0.4258 | – | – | – |
| LSTM-ALO [54] | 0.62 | – | – | – |

**Table 9**
Best hyperparameter combination.

| Parameters | CNN7 | B-CNN7 |
|---|---|---|
| Dropout rate1 | 0.25 | 0.16 |
| Dropout rate2 | 0.5 | 0.20 |
| Learning rate | 0.001 | 0.0012 |
| Batch size | 100 | 89 |

**Table 10**
Comparison of hyperparameter optimization results.

| Feature | Accuracy | Precision | Recall | F1 score |
|---|---|---|---|---|
| CNN7 | 0.62 | 0.71 | 0.80 | 0.75 |
| B-CNN7 | **0.67** | **0.73** | **0.86** | **0.79** |

optimization with CNNs, combined with gold price, gold volatility index, crude oil price, and crude oil price volatility index information achieved a good prediction of the future stock market trends. Therefore, the models used in the subsequent experiments adopts BO to find the optimal hyperparameter values. Table 10 provides a comparison of the hyperparameter optimization results.

*4.1.4. S&P 500 test set prediction accuracy with time period*

In Table 11, the time period field indicates the number of days observed in the past, and T+1 and T+5 indicate that the predicted time in the future was one and five days later, respectively. Furthermore, it may be observed from this table that CNN4's short-term forecasting ability increased with the number of days referenced in the past, and that CNN7 was more accurate

**Table 11**
CNN4 vs. CNN7: Accuracy comparison on two-category classification.

| Time period | Model | T + 1 | T + 5 | T + 10 | T + 15 | T + 20 |
|---|---|---|---|---|---|---|
| 10 | CNN4 | **0.47** | 0.50 | 0.50 | (X) | (X) |
| | CNN7 | 0.6(O) | 0.39 | 0.46 | (X) | (X) |
| 15 | CNN4 | **0.47** | 0.48 | 0.50 | 0.54 | (X) |
| | CNN7 | 0.56(O) | 0.39 | 0.57 | 0.43 | (X) |
| 20 | CNN4 | **0.51** | 0.49 | 0.60 | 0.52 | 0.51 |
| | CNN7 | 0.6(O) | **0.50** | **0.67** | **0.68** | **0.69** |
| 25 | CNN4 | **0.54** | 0.47 | 0.42 | 0.45 | 0.45 |
| | CNN7 | 0.69(O) | **0.44** | **0.54** | **0.68** | **0.73** |
| 30 | CNN4 | **0.59** | 0.45 | 0.54 | 0.42 | 0.47 |
| | CNN7 | 0.69(O) | 0.44(O) | **0.58** | **0.65** | **0.70** |

Remarks: (O) is the data of overfitting transition learning training set. (X) indicates an undesired situation.

for the future trend of the forecast target time. This behavior is attributed to the marking class labeling corresponding to the feature image. When a technical index was used as feature data (CNN4), the index features were either the same or similar, and the multi-category labels are the same at T+1; therefore, the model can learn normally and provide a good judgment. The CNN7 data were related to oil and gold; their characteristics were not immediately reflected in the market, resulting in oil and gold being associated with the same feature image. This causes the mark to be either 0 or 1 at T+1. This renders the model incapable of possessing good judgment criteria. In contrast, it is easy to obtain a unified standard for corresponding marks in the same feature image at T+10, T+15, and T+20, particularly because gold and oil do more significantly affect stock prices in the long run. Table 12 shows CNN4's confusion matrix for the two-category classification method (time period: 20, model: CNN4, T+10).

In summary, such a two-category labeling method does not help in buying low and selling high in stock trading. For example, consider the following. If a stock is predicted to rise after 10 days, it must be held for 10 trading days after buying. However, it is unknown if the 10th coming trading day would have the best transaction price. It is possible that selling on the 11th or 12th day can sell the stock at higher price; therefore, this

**Table 12**
Two-category classification confusion matrix (Time period: 20, Model: CNN4, T + 10).

| Total population | | Predicted | |
|---|---|---|---|
| | | Up | Down |
| Real | Up | 110 | 62 |
| | Down | 33 | 31 |

**Table 13**
Buy-Hold-Sell scale.

| Window size | Buy | Hold | Sell |
|---|---|---|---|
| 7 | 1 | 8.45 | **1.25** |
| 9 | 1 | 11.95 | **1.25** |
| 11 | 1 | 14.66 | 1.11 |
| 13 | 1 | 17.60 | 1.09 |
| 15 | 1 | **23.79** | 1.01 |

**Table 15**
Comparison of accuracy rate and rate of return for CNN4/7/8.

| Time period | Model | Accuracy | 2019 Rate of return |
|---|---|---|---|
| 10 | CNN4 | **0.90** | 8.67% |
| | CNN7 | 0.58 | −0.5% |
| | CNN8 | 0.88 | 10.87% |
| 20 | CNN4 | **0.93** | 12.08% |
| | CNN7 | 0.70 | −2.57% |
| | CNN8 | 0.78 | **13.23%** |

**Table 16**
Comparison of LSTM and CNN rate of return.

| Model | Buy | Hold | Sell | 2019 Rate of return |
|---|---|---|---|---|
| CNN8 | 37 | 174 | 34 | **13.23%** |
| LSTM8 | 22 | 188 | 35 | 11.06% |
| CNN4 | 14 | 220 | 11 | 12.08% |
| LSTM4 | 31 | 173 | 41 | 7.81% |

study additionally considers the abovementioned three-category approach to mark the best buying, selling, and holding points. The model can predict future trading signals and conclude whether a stock should be bought, sold, or held the next day.

### 4.2. Experimental results of three-category classification

We use the three-category classification (buy, sell, hold) to mark the data. According to the images generated from past data, it is predicted whether a stock should be bought, sold, or held the next day. The dataset recording the S&P 500 (GSPC), with a total ten-year interval of data from 2010 to 2019 (Table 2). Of these data, the data from 2019 constituted the test data by virtue of being outside the sample, and the data from 2010 to 2018 were split at a ratio of 8:2. Here, 80% of the data were used as training data, and the remaining 20% were used as verification data within the sample. Further discussion on this topic is divided into the following five subsections.

#### 4.2.1. Discussion on the three-category marking method

Table 13 lists the buying, selling, and holding ratios in different window sizes. As the window size was reduced, the number of buying and selling points increased, and vice versa. Table 14 lists the rate of return from 2010 to 2019, rate of return for 2019 alone, and annualized rate of return. The starting capital for calculating this rate of return was USD $10,000, and each transaction incurred a handling fee of 0.09%. The handling fee is a reference to that charged by the eToro trading platform. According to the backtest results in Table 14, it mat be observed that the window size applicable to each stock was different. We used a window size of 11 days.

#### 4.2.2. Testing and evaluation of feature correct rate and rate of return

Table 15 presents the accuracy rates of different models using the test data (out-of-sample) rate of return of each model for the data of 10 or 20 days prior to each observation. The prediction goal describes the next days' recommended action: buy, sell, or hold. CNN4s' characteristic data included only a financial indicator, and CNN7 was a combination of gold and oil values (Table 3). Irrespective of the perspective of accuracy or rate of return, each observation of 20 days was better than the corresponding 10-day observation. CNN7 only observed oil and gold, and the short-term response was not apparent, thereby resulting in a classification accuracy rate and backtest rate of return that were significantly lower than the reference financial indicators. Therefore, CNN8 was added for comparison with CNN4. The characteristic data of CNN8 comprise financial indicators and the prices of crude oil and gold. Although CNN4 was superior to CNN8 in terms of accuracy, CNN8 was superior to CNN4 and CNN7 in terms of the rate of return.

#### 4.2.3. Three-category test set evaluation model for validity and rate of return

Table 16 presents a comparison of the effectiveness of the three-category classification model. Compared to the two-category classification model, the three-category model (i) has identical input feature data and variables, (ii) only replaces the deep learning model, and (iii) also performs hyperparameter selection using BO. On the test set (outside the sample), the rate of return of CNN8 was superior to that of LSTM8. From the values of ROI in Table 16, it may be observed that the CNN4 or CNN8 models were superior to the LSTM4 or LSTM8 models.

#### 4.2.4. Comparison of buying over varying numbers of trades

Herein, it is considered that in actual investment strategies, investors do not invest all their funds into the market at once. Suppose there is an investment of US$ 10,000. Therefore, in the case of the three-category mark, we tested whether stepwise buying could achieve an improved ROI. The results in Table 17 indicate that, in the case of real, marked buying and selling points,

**Table 14**
S&P 500 real rate of return based on different window sizes.

| Window size | 2010 to 2019 rate of return | 2019 rate of return | Annual rate of return |
|---|---|---|---|
| 3 | −17.54% | 11.21% | −0.191% |
| 5 | −23.25% | 2.51% | −2.61% |
| 7 | −12.59% | 5.86% | −1.34% |
| 9 | 19.91% | 4.46% | 1.83% |
| 11 | 32.79% | 16.36% | 2.88% |
| 13 | 33.51% | 18.11% | 2.93% |
| 15 | 51.37% | 18.11% | 4.23% |
| 17 | 71.18% | **19.45%** | 5.52% |
| 19 | **77.58%** | 18.07% | **5.91%** |

**Table 17**
CNN8 investment test results (buying over varying numbers of trades).

| ROI (buying for different numbers of times) | 1 | 2 | 3 | 5 |
|---|---|---|---|---|
| 2019 Real ROI | **16.36%** | 10.28% | 6.31% | 1.58% |
| 2019 Predict ROI | **13.23%** | 3.21% | 11.63% | 3.12% |

**Table 18**
Performance comparison of CNN8, LSTM8, ETFs [1] and Dow-30.

| Data<br>Model | S&P 500<br>CNN8 | S&P 500<br>LSTM8 | ETFs<br>CNN-TA | Dow-30<br>CNN-TA |
|---|---|---|---|---|
| Total accuracy | 0.78 | **0.80** | 0.62 | 0.58 |
| Precision (Buy) | 0.30 | **0.32** | 0.19 | 0.22 |
| Recall (Buy) | **0.92** | 0.58 | 0.72 | 0.80 |
| F1 score (Buy) | **0.45** | 0.41 | 0.30 | 0.34 |
| Precision (Hold) | **0.98** | 0.95 | 0.95 | 0.95 |
| Recall (Hold) | 0.77 | **0.81** | 0.61 | 0.55 |
| F1 score (Hold) | 0.86 | **0.88** | 0.75 | 0.70 |
| Precision (Sell) | **0.29** | 0.26 | 0.15 | 0.18 |
| Recall (Sell) | 0.77 | 0.69 | 0.66 | **0.81** |
| F1 score (Sell) | **0.43** | 0.37 | 0.24 | 0.29 |

step-by-step buying did not increase the rate of return, primarily because there were very few real, marked buying and selling points in 2019. In the proposed method, CNN8's forecast for 2019 buying and selling points still provided the best investment rate of return for a one-time investment. The strategy of buying over varying numbers of trades can be categorized as buying a stock one, two, three, or five times. The cases of three or five times accidentally resulted in a better rate of return than the real marking method.

#### 4.2.5. Comparison of results from related literature

Table 18 presents the performance of various models, including the proposed CNN8, LSTM8, on various investment funds including ETFs (XLF, XLU, QQQ, SPY, XLP, EWZ, EWH) from the experiments of [1], XLY, XLE), and a Dow-30 evaluation. The experimental results demonstrate that the recall values of the buying and selling points were superior to those of the holding points; however, the precision of the buying and selling points was inferior to those of the holding points. However, for stock trading, accurate entry and exit points (buy, sell) are essential to obtain a good ROI. The results in Table 15 indicate that when more characteristics such as oil and gold are considered, backtesting the ROI becomes more useful than simply referring to the financial indicators. CNN8 was able to find nearly all the best entry and exit points. Table 18 shows that the F1 scores of CNN8 in buy, hold, and sell were better than those of other combinations. Furthermore, in the case of CNN8, recall was best on the buy function. In summary, the rate of ROI was significantly affected by the buying and selling points being of high or low quality. It can be concluded that, to obtain a good ROI, it may be expected to be effective to refer to oil prices, oil fluctuation indices, gold prices, and gold price fluctuation indices as input characteristics.

## 5. Conclusions, discussions, limitations and recommendations

This section presents our final conclusions along with some discussions as well as a description of the limitations of our proposed method and recommendations for its implementation. The conclusion explains the experimental results of the two-category and three-category classification models. The final section provides suggestions for future research.

### 5.1. Conclusions

This study covers a variety of specific points, including methods of inputting characteristic data, approaches to using financial technical indicators and gold price, gold volatility index, crude oil price, crude oil price volatility index information, and finally the construction of two neural network learning models to predict whether a future segment of the S&P 500 stock index would move up or down. The three-category method was used to find corresponding trading opportunities, and the investment strategy of buying low and selling high was used to backtest the rate of return. The research conclusions of the two-category and the three-category experiment results are discussed in separate subsections, by observing the classification accuracy and investment performance and considering the suitability of individual stocks.

The results of this study show that gold prices, gold volatility indices, crude oil prices, and crude oil price volatility indices can be used as input features for deep learning models. Compared to the results obtained using financial technology indicators, the predictions of rise or fall in stock prices after a certain period of time in the future were shown to be more accurate. If the stock is predicted to rise in the future, then it can be bought now, and vice versa. In this classification, a model accuracy rate of up to 67% was attained by the proposed method. The results in Table 11 indicate that the model can be applied for the period of the next 10 to 20 days. In this classification, the results of the CNN and LSTM were indistinguishable from each other. Additionally, we confirmed that using BO to optimize hyperparameters can increase the accuracy of the model classification results.

The forecast goal of this study in the three-category classification problem was whether to buy, sell, or hold a stock the next day. The gold price, gold volatility index, crude oil price, and crude oil price volatility index data were less accurate when used at t-1 (CNN7). Therefore, we discuss the use of gold price, gold volatility index, crude oil price, and crude oil price volatility index information, and financial indicators as input features to compare with the input features of simple financial technical indicators.

We found that in the case of CNNs, when the gold price, gold volatility index, crude oil price, and crude oil price volatility index are used as the input features of the model, the proposed three-category CNN classified the buying and selling points more accurately than merely using simple financial indicators. The experiments in Section 3.2.2 and Table 16 assumed the case of incremental investment. The maximum ROI of the purchases was 13.23% in the case of a one-time purchase, and it could reach up to 11.63% in the case of three purchases. This model focuses on the buying and selling points predicted outside the sample in 2019. The measured ROI of CNN8 could reach up to 13.23%, which is superior to 12.08% for CNN4 and 11.06% for LSTM8.

The above conclusion is true for S&P 500, and this study applied the three-category classification method to multiple S&P 500 stock companies. It can be observed from the research results that when the input characteristics refer to gold price, gold fluctuation index, crude oil price, and crude oil price volatility index information, the prediction results were observed to be appropriate for the companies in the semiconductor, petroleum and energy, and automotive industries. However, we consider it more suitable to use pure financial technology indicators for companies in the apparel, fast food, and copy processing sectors.

### 5.2. Discussions

The question of whether regular fixed investment or one-time investment performs better has always been a controversial topic. Calculated from the actual ROI in 2019, the one-time investment method had a return on investment of 13.36%, which was better

than the 10.28% of the two-stage investment and 6.31% of the three-stage investment. The worst performance was 1.58% of the five-stage investment. In Section 4.2.4, through the investment methods suggested in this research, a one-time investment could also achieve a return on investment of 13.23%, even if the return on investment of the three-stage investment and the five-stage investment was better than the actual 2019 marking data.

Another controversial topic concerns the S&P 500, ETF, and Dow Jones 30 stock baskets. Which one may be expected to perform better? From Table 18, in terms of F1 scores, the S&P 500 performed better than ETFs and Dow30 using the proposed method.

### 5.3. Limitations

A significant limitation of this research is that the sample data were collected from 2010 to 2018 for deep learning training and from 2019 for testing. The predicted target was also limited to S&P 500 for the corresponding years. If the sample data were changed, the entire experiment would have to be repeated, and the experimental results might differ.

### 5.4. Research recommendations

The neural network model constructed in this research can be considered relatively mature to be suitable for application tor stock price trends and classified investment. The selection of model variables and the use of BO for hyperparameters are also relatively mature techniques; however, the number of neurons per layer in the model can be further optimized. A further disadvantage is that the BO process is usually time-consuming. In the future, the use of BO and hyperband (BOHB) [57] can be considered while optimizing the parameters. BOHB is an improvement over BO. The research results showed that BOHB can be used in parallel.

In recent years, the use of unsupervised learning networks, such as generative adversarial networks and self-organizing mapping networks, has gained increasing attention. Subsequent research may be based on the characteristic data obtained in this study, and the use of unsupervised learning may also be explored. Moreover, in the current study, only gold price, gold volatility index, crude oil price, and crude oil price volatility index data were considered. In the future, interest rates, exchange rate, other futures prices, and volatility indices may also be considered.

## CRediT authorship contribution statement

**Yu-Chen Chen:** Conceptualization, Methodology, Software, Data curation, Writing – original draft. **Wen-Chen Huang:** Supervision, Validation, Writing – review & editing.

## Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

## References


[1] O.B. Sezer, A.M. Ozbayoglu, Algorithmic financial trading with deep convolutional neural networks: Time series to image conversion approach, Appl. Soft Comput. 70 (2018) 525–538.

[2] H.S. Sim, H.I. Kim, J.J. Ahn, Is deep learning for image recognition applicable to stock market prediction? Complexity 2019 (2019).

[3] H. Chung, K.S. Shin, Genetic algorithm-optimized multi-channel convolutional neural network for stock market prediction, Neural Comput. Appl. (ISSN: 0941-0643) 32 (12) (2020) 7897–7914, http://dx.doi.org/10.1007/s00521-019-04236-3, URL <Go to ISI>://WOS:000540259800022.

[4] M.A. El-Rashidy, A novel system for fast and accurate decisions of gold-stock markets in the short-term prediction, Neural Comput. Appl. (ISSN: 0941-0643) (2020) http://dx.doi.org/10.1007/s00521-020-05019-x, URL <Go to ISI>://WOS:000534406400001.

[5] K.K. Gokmenoglu, N. Fazlollahi, The interactions among gold, oil, and stock market: evidence from S&P500, Procedia Econ. Finance 25 (Supp. C) (2015) 478–488.

[6] C.L. Xiao, W.L. Xia, J.J. Jiang, Stock price forecast based on combined model of ARI-MA-LS-SVM, Neural Comput. Appl. (ISSN: 0941-0643) 32 (10) (2020) 5379–5388, http://dx.doi.org/10.1007/s00521-019-04698-5, URL <Go to ISI>://WOS:000529745200007.

[7] I.R. Parray, S.S. Khurana, M. Kumar, A.A. Altalbe, Time series data analysis of stock price movement using machine learning techniques, Soft Comput. (ISSN: 1432-7643) (2020) http://dx.doi.org/10.1007/s00500-020-04957-x, URL <Go to ISI>://WOS:000528126600001.

[8] J.S. Cao, J.H. Wang, Exploration of stock index change prediction model based on the combination of principal component analysis and artificial neural network, Soft Comput. (ISSN: 1432-7643) 24 (11) (2020) 7851–7860, http://dx.doi.org/10.1007/s00500-019-03918-3, URL <Go to ISI>://WOS:000530547900003.

[9] W. Chen, M.R. Jiang, C. Jiang, Constructing a multilayer network for stock market, Soft Comput. (ISSN: 1432-7643) 24 (9) (2020) 6345–6361, http://dx.doi.org/10.1007/s00500-019-04026-y, URL <Go to ISI>://WOS:000523742000011.

[10] J.H. Zhao, D.L. Zeng, S. Liang, H.L. Kang, Q.M. Liu, Prediction model for stock price trend based on recurrent neural network, J. Ambient Intell. Humaniz. Comput. (ISSN: 1868-5137) (2020) http://dx.doi.org/10.1007/s12652-020-02057-0, URL <Go to ISI>://WOS:000532111700003.

[11] J.W. Long, Z.P. Chen, W.B. He, T.Y. Wu, J.T. Ren, An integrated framework of deep learning and knowledge graph for prediction of stock price trend: An application in Chinese stock exchange market, Appl. Soft Comput. (ISSN: 1568-4946) 91 (2020) http://dx.doi.org/10.1016/j.asoc.2020.106205, URL <Go to ISI>://WOS:000535478600016.

[12] F. Kamalov, Forecasting significant stock price changes using neural networks, Neural Comput. Appl. (2020) 1–13.

[13] G.L. Tang, F. Chiclana, P.D. Liu, A decision-theoretic rough set model with q-rung orthopair fuzzy information and its application in stock investment evaluation, Appl. Soft Comput. (ISSN: 1568-4946) 91 (2020) http://dx.doi.org/10.1016/j.asoc.2020.106212, URL <Go to ISI>://WOS:000535478600014.

[14] B. Cao, J.W. Zhao, Z.H. Lv, Y. Gu, P. Yang, S.K. Halgamuge, Multiobjective evolution of fuzzy rough neural network via distributed parallelism for stock prediction, IEEE Trans. Fuzzy Syst. (ISSN: 1063-6706) 28 (5) (2020) 939–952, http://dx.doi.org/10.1109/tfuzz.2020.2972207, URL <Go to ISI>://WOS:000532212600012.

[15] Y.D. Ge, J.N. Qiu, Z.Y. Liu, W.J. Gu, L.W. Xu, Beyond negative and positive: Exploring the effects of emotions in social media during the stock market crash, Inf. Process. Manage. (ISSN: 0306-4573) 57 (4) (2020) http://dx.doi.org/10.1016/j.ipm.2020.102218, URL <Go to ISI>://WOS:000531082800007.

[16] Q.F. Xu, L.K. Wang, C.X. Jiang, Y.Z. Liu, A novel (U)MIDAS-SVR model with multi-source market sentiment for forecasting stock returns, Neural Comput. Appl. (ISSN: 0941-0643) 32 (10) (2020) 5875–5888, http://dx.doi.org/10.1007/s00521-019-04063-6, URL <Go to ISI>://WOS:000529745200045.

[17] A. Loginov, M. Heywood, G. Wilson, Stock selection heuristics for performing frequent intraday trading with genetic programming, Genet. Program. Evol. Mach. (ISSN: 1389-2576) (2020) http://dx.doi.org/10.1007/s10710-020-09390-5, URL <Go to ISI>://WOS:000529737700001.

[18] K. Koppula, B.S. Kedukodi, S.P. Kuncham, Markov frameworks and stock market decision making, Soft Comput. (ISSN: 1432-7643) (2020) http://dx.doi.org/10.1007/s00500-020-04950-4, URL <Go to ISI>://WOS:000534315200003.

[19] Y. Karaca, Y.D. Zhang, K. Muhammad, Characterizing complexity and self-similarity based on fractal and entropy analyses for stock market forecast modelling, Expert Syst. Appl. (ISSN: 0957-4174) 144 (2020) http://dx.doi.org/10.1016/j.eswa.2019.113098, URL <Go to ISI>://WOS:000514218700019.

[20] H.M. Kim, S. Jung, J. Park, The impact of past performance on information valuation in virtual communities: Empirical study in online stock message boards, Inf. Process. Manage. (ISSN: 0306-4573) 54 (4) (2018) 740–753, http://dx.doi.org/10.1016/j.ipm.2017.10.007, URL <Go to ISI>://WOS:000434743700016.

[21] M. Ananthi, K. Vijayakumar, Stock market analysis using candlestick regression and market trend prediction (CKRM), J. Ambient Intell. Humaniz. Comput. (ISSN: 1868-5137) (2020) http://dx.doi.org/10.1007/s12652-020-01892-5, URL <Go to ISI>://WOS:000528307400003.

[22] J.T. Han, Z.P. Ge, Effect of dimensionality reduction on stock selection with cluster analysis in different market situations, Expert Syst. Appl. (ISSN: 0957-4174) 147 (2020) http://dx.doi.org/10.1016/j.eswa.2020.113226, URL <Go to ISI>://WOS:000521117700030.

[23] I.K. Nti, A.F. Adekoya, B.A. Weyori, A systematic review of fundamental and technical analysis of stock market predictions, Artif. Intell. Rev. (ISSN: 0269-2821) 53 (4) (2020) 3007–3057, http://dx.doi.org/10.1007/s10462-019-09754-z, URL <Go to ISI>://WOS:000524414900021.
[24] P.W. Wang, L. Zong, Y. Ma, An integrated early warning system for stock market turbulence, Expert Syst. Appl. (ISSN: 0957-4174) 153 (2020) http://dx.doi.org/10.1016/j.eswa.2020.113463, URL <Go to ISI>://WOS:000533513600012.
[25] B. Lutz, N. Prollochs, D. Neumann, Predicting sentence-level polarity labels of financial news using abnormal stock returns, Expert Syst. Appl. (ISSN: 0957-4174) 148 (2020) http://dx.doi.org/10.1016/j.eswa.2020.113223, URL <Go to ISI>://WOS:000525814900004.
[26] T. Jerez, W. Kristjanpoller, Effects of the validation set on stock returns forecasting, Expert Syst. Appl. (ISSN: 0957-4174) 150 (2020) http://dx.doi.org/10.1016/j.eswa.2020.113271, URL <Go to ISI>://WOS:000528193700013.
[27] B.L. Chen, J.D. Zhong, Y.Y. Chen, A hybrid approach for portfolio selection with higher-order moments: Empirical evidence from Shanghai Stock Exchange, Expert Syst. Appl. (ISSN: 0957-4174) 145 (2020) http://dx.doi.org/10.1016/j.eswa.2019.113104, URL <Go to ISI>://WOS:000518692500007.
[28] A. Altan, O. Aslan, R. Hacıoğlu, Real-time control based on NARX neural network of hexarotor UAV with load transporting system for path tracking, in: 2018 6th International Conference on Control Engineering Information Technology (CEIT), 2018, pp. 1–6, http://dx.doi.org/10.1109/CEIT.2018.8751829.
[29] A. Altan, Performance of metaheuristic optimization algorithms based on swarm intelligence in attitude and altitude control of unmanned aerial vehicle for path following, in: 2020 4th International Symposium on Multidisciplinary Studies and Innovative Technologies (ISMSIT), 2020, pp. 1–6, http://dx.doi.org/10.1109/ISMSIT50672.2020.9255181.
[30] A. Altan, S. Karasu, Recognition of COVID-19 disease from X-ray images by hybrid model consisting of 2D curvelet transform, chaotic salp swarm algorithm and deep learning technique, Chaos Solitons Fractals (ISSN: 0960-0779) 140 (2020) 110071, http://dx.doi.org/10.1016/j.chaos.2020.110071, URL https://www.sciencedirect.com/science/article/pii/S0960077920304689.
[31] D.M. Nelson, A.C. Pereira, R.A. de Oliveira, Stock market's price movement prediction with LSTM neural networks, in: 2017 International Joint Conference on Neural Networks (IJCNN), IEEE, 2017, pp. 1419–1426.
[32] A. Altan, S. Karasu, E. Zio, A new hybrid model for wind speed forecasting combining long short-term memory neural network, decomposition methods and grey wolf optimizer, Appl. Soft Comput. (ISSN: 1568-4946) 100 (2021) 106996, http://dx.doi.org/10.1016/j.asoc.2020.106996, URL https://www.sciencedirect.com/science/article/pii/S1568494620309352.
[33] F. Nogueira, Bayesian optimization: Open source constrained global optimization tool for Python, 2014, URL https://github.com/fmfn/BayesianOptimization.
[34] H. Li, Y. Shen, Y. Zhu, Stock price prediction using attention-based multi-input LSTM, in: Asian Conference on Machine Learning, 2018, pp. 454–469.
[35] J. Patel, S. Shah, P. Thakkar, K. Kotecha, Predicting stock and stock price index movement using trend deterministic data preparation and machine learning techniques, Expert Syst. Appl. 42 (1) (2015) 259–268.
[36] S. Karasu, A. Altan, S. Bekiros, W. Ahmad, A new forecasting model with wrapper-based feature selection approach using multi-objective optimization technique for chaotic crude oil time series, Energy 212 (C) (2020) http://dx.doi.org/10.1016/j.energy.2020.118, URL https://ideas.repec.org/a/eee/energy/v212y2020ics0360544220318570.html.
[37] Mars. URL https://github.com/mars-project/mars.
[38] pyearth. URL https://contrib.scikit-learn.org/py-earth/content.html.
[39] J.H. Friedman, Multivariate adaptive regression splines, Ann. Statist. (1991) 1–67.
[40] J.H. Friedman, C.B. Roosen, An Introduction to Multivariate Adaptive Regression Splines, Sage Publications Sage CA, Thousand Oaks, CA, 1995.
[41] GMDhPy. URL https://github.com/kvoyager/GmdhPy.
[42] J. Schmidhuber, Deep learning in neural networks: An overview, Neural Netw. 61 (2015) 85–117.
[43] J.-A. Mueller, F. Lemke, Self-Organising Data Mining: Extracting Knowledge from Data, Libri Books on Demand, 2000.
[44] A. Ivakhnenko, G. Ivakhnenko, The review of problems solvable by algorithms of the group method of data handling (GMDH), in: Pattern Recognition and Image Analysis C/C of Raspoznavaniye Obrazov i Analiz Izobrazhenii, Vol. 5, Citeseer, 1995, pp. 527–535.
[45] L. Anastasakis, N. Mort, The Development of Self-Organization Techniques in Modelling: A Review of the Group Method of Data Handling (GMDH), RESEARCH REPORT-UNIVERSITY of SHEFFIELD DEPARTMENT of AUTOMATIC CONTROL and SYSTEMS ENGINEERING, 2001.
[46] N.K. Kasabov, Q. Song, DENFIS: dynamic evolving neural-fuzzy inference system and its application for time-series prediction, IEEE Trans. Fuzzy Syst. 10 (2) (2002) 144–154.
[47] L.S. Riza, C.N. Bergmeir, F. Herrera Triguero, J.M. Benítez Sánchez, et al., frbs: Fuzzy Rule-Based Systems for Classification and Regression in R, American Statistical Association, 2015.
[48] L.S. Riza, C. Bergmeir, F. Herrera, J.M. Benítez, Learning from data using the R package "FRBS", in: 2014 IEEE International Conference on Fuzzy Systems (FUZZ-IEEE), IEEE, 2014, pp. 2149–2155.
[49] S. Mirjalili, The ant lion optimizer, Adv. Eng. Softw. 83 (2015) 80–98.
[50] S. Mirjalili, P. Jangir, S. Saremi, Multi-objective ant lion optimizer: a multi-objective optimization algorithm for solving engineering problems, Appl. Intell. 46 (1) (2017) 79–95.
[51] E. Ali, S. Abd Elazim, A. Abdelaziz, Ant lion optimization algorithm for optimal location and sizing of renewable distributed generations, Renew. Energy 101 (2017) 1311–1324.
[52] H.M. Zawbaa, E. Emary, B. Parv, Feature selection based on antlion optimization algorithm, in: 2015 Third World Conference on Complex Systems (WCCS), IEEE, 2015, pp. 1–7.
[53] L. Abualigah, M. Shehab, M. Alshinwan, S. Mirjalili, M. Abd Elaziz, Ant lion optimizer: a comprehensive survey of its variants and applications, Arch. Comput. Methods Eng. 28 (3) (2021) 1397–1416.
[54] X. Yuan, C. Chen, X. Lei, Y. Yuan, R.M. Adnan, Monthly runoff forecasting based on LSTM–ALO model, Stoch. Environ. Res. Risk Assess. 32 (8) (2018) 2199–2212.
[55] Ant Lion Optimizer (ALO). URL https://github.com/zhaoxingfeng/ALO.
[56] T.A. Rashid, P. Fattah, D.K. Awla, Using accuracy measure for improving the training of LSTM with metaheuristic algorithms, Procedia Comput. Sci. 140 (2018) 324–333.
[57] S. Falkner, A. Klein, F. Hutter, BOHB: Robust and efficient hyperparameter optimization at scale, in: Proceedings of the 35th International Conference on Machine Learning, 2018, pp. 1436–1445.